Procesamiento de Imágenes

Escuela de Informática y Telecomunicaciones

Universidad Diego Portales

TAREA 2

Profesor: Jorge E. Zambrano.

Fecha de entrega: domingo 01 de noviembre, 23:59 (en pareja o individual)

1. Objetivo

La inspección del estado de un pavimento se hace hoy con vehículos que fotografían la calzada de forma continua. Localizar automáticamente las grietas en esas imágenes es difícil: la grieta ocupa una fracción mínima de la imagen, su contraste contra el asfalto es bajo, y la textura del propio asfalto tiene la misma escala espacial que la grieta.

En esta tarea construirán, paso a paso, un detector de grietas que encadena operadores de las tres unidades del curso: filtrado espacial, detección de bordes y morfología matemática. En cada paso medirán cuánto aporta la etapa que agregan, y al final analizarán esos resultados.

2. Descripción

2.1 Dataset

Se trabajará con CrackForest [1], un conjunto de 118 imágenes de pavimento urbano de 320 × 480 píxeles, cada una con una máscara binaria anotada a nivel de píxel. El cuaderno base lo descarga automáticamente desde GitHub:

```
git clone --depth 1 https://github.com/cuilimeng/CrackForest-dataset.git
```

Nota: la carpeta `image/` contiene 155 archivos, pero solo 118 tienen máscara. El cuaderno base ya construye la lista `IDS` a partir de `groundTruth/`.

2.2 Cuaderno base

Se entrega el cuaderno `Tarea2_base.ipynb`. Trae implementado lo siguiente, que **no deben modificar**:

Función	Retorna
cargar_par(nombre)	(imagen en grises, máscara binaria)
metricas(pred, gt)	{"IoU", "P", "R", "F1"}
metricas_con_tolerancia(pred, gt, tol)	{"P_tol", "R_tol", "F1_tol"}
evaluar_por_imagen(segmentador, ids)	DataFrame con una fila por imagen
evaluar_conjunto(segmentador, ids)	promedio de todas las métricas
tabla(resultados)	DataFrame con IoU, P, R, F1, F1_tol
mostrar(imagenes, titulos, cols)	grilla de visualización
superponer(img, pred, gt)	verde = TP, rojo = FP, azul = FN
FILTROS	diccionario con los seis filtros de la Parte 1

El cuaderno está ordenado según los pasos de este enunciado. Cada función que deben implementar aparece con su firma y `raise NotImplementedError`. Sus nombres y argumentos deben quedar exactamente como están. En `segmentar_canny` (Paso 5) y `postprocesar` (Paso 8) una parte del código ya viene escrita, marcada como "Entregado": no la modifiquen. Los resultados y su análisis no van en el cuaderno, sino en el informe en PDF (Secciones 7 y 8).

2.3 Convenciones y métricas

Toda función de segmentación recibe una imagen en escala de grises (`uint8`) y devuelve una máscara binaria del mismo tamaño, de tipo `uint8` y con valores en {0, 1}. Una máscara con valores 0 y 255 no cumple el contrato: `evaluar_por_imagen` la rechaza con un error. Tenga presente que `cv2.Canny`, `cv2.floodFill` y `cv2.threshold` con valor máximo 255 producen 0-255, por lo que hay que convertir el resultado antes de devolverlo.

En promedio, solo el 1.6 % de los píxeles de una imagen pertenece a una grieta. Por eso la exactitud (*accuracy*) es inútil: una máscara vacía obtendría más de 98 %. Se evalúa con IoU, precisión (P), recall (R) y F1, que ignoran los verdaderos negativos. La variante con tolerancia (tol = 2) admite un desajuste de hasta dos píxeles entre la predicción y la referencia, porque la anotación manual de una estructura de tres píxeles de ancho no es exacta.

3. Parte 1. Filtrado

La textura del asfalto es ruido de alta frecuencia cuya escala espacial coincide con el ancho de la grieta. En esta parte medirán qué filtro de suavizado conviene, con dos criterios: el CNR y el efecto del filtro sobre la segmentación final.

¿Qué mide el CNR y cómo se interpreta?

La **relación contraste-ruido** (CNR, *contrast-to-noise ratio*) compara la diferencia de intensidad entre la grieta y el fondo con la variabilidad del fondo:

$$\mathrm{CNR} = \frac{\left|\mu_{\text{grieta}} - \mu_{\text{fondo}}\right|}{\sigma_{\text{fondo}}}$$

- **Numerador (contraste):** cuántos niveles de gris más oscura es, en promedio, la grieta que el asfalto.
- **Denominador (ruido):** cuánto varía el asfalto por su propia textura, medido con la desviación estándar de los píxeles de fondo.
- **Cociente:** cuántas desviaciones estándar del fondo separan a la grieta del fondo. Es adimensional, por lo que permite comparar imágenes y filtros distintos.

Interpretación. Un CNR cercano a 1 indica que la diferencia entre grieta y fondo es del mismo tamaño que las fluctuaciones de la textura: un píxel oscuro aislado puede ser tanto grieta como un poro del asfalto, y la intensidad por sí sola no permite distinguirlos. Mientras mayor sea el CNR, más separables son las dos clases. Como referencia, el criterio de Rose indica que se necesita un CNR de entre 3 y 5 para detectar un objeto de forma confiable [5]. En CrackForest, las imágenes sin filtrar tienen un CNR cercano a 2, lo que explica en buena parte la dificultad del problema.

Efecto de un filtro. Un filtro de suavizado puede subir el CNR porque reduce la textura del fondo (baja el denominador). Pero si su ventana es más ancha que la grieta, también mezcla la grieta con el asfalto que la rodea y reduce el contraste (baja el numerador). El CNR sube solo si el ruido baja más que el contraste.

Limitación. El CNR es un promedio global de dos poblaciones de píxeles: no considera la forma ni la posición de la grieta. Un filtro con mejor CNR no garantiza una mejor segmentación. Por eso, en el Paso 3, el CNR se compara con el F1 obtenido al segmentar con cada filtro.

Paso 1. Implementar `contraste_grieta_fondo(img, mascara)`

- Convierta la imagen a `np.float64` y la máscara a booleano (`g = mascara.astype(bool)`).
- Calcule la media de los píxeles de grieta (`x[g]`), la media y la desviación estándar de los píxeles de fondo (`x[~g]`).
- Devuelva el CNR como un escalar, usando el valor absoluto de la diferencia de medias.

Paso 2. Implementar `segmentar_blackhat_filtro(img, filtro, k_bh=21, k_sigma=3.0)`

Esta función es la ruta de segmentación que usarán para medir el efecto de cada filtro, y la reutilizarán en la ruta C (Paso 6) y en el pipeline final (Paso 10).

Idea. La grieta es un valle oscuro y angosto, de 3 a 5 píxeles de ancho, dentro de un asfalto más claro. La transformada **black-hat**, cierre(f) − f, aprovecha esa forma: el cierre con un elemento de `k_bh` × `k_bh` rellena todo valle más angosto que el elemento con el nivel del asfalto que lo rodea, pero casi no modifica las zonas planas. Al restar la imagen original, el asfalto queda cerca de cero y la grieta aparece como una línea brillante, cuya intensidad es la profundidad del valle.

Umbral adaptativo. Como la respuesta es casi nula en la mayor parte de la imagen, su media y su desviación estándar describen al fondo. El umbral media + `k_sigma` · desviación conserva solo los valores atípicos, que corresponden a la grieta, y se ajusta solo a cada imagen: no depende de si es más clara, más oscura o tiene más textura.

Por qué el filtro es un argumento. Los poros del asfalto también son pequeños valles oscuros y el black-hat también responde a ellos. El filtro de suavizado decide cuántos poros sobreviven y cuánto se conserva la grieta. Recibirlo como función permite cambiar solo el suavizado y dejar fijo todo lo demás, de modo que las diferencias de F1 de la Tabla 1 se deban únicamente al filtro. Por la misma razón, la función no aplica post-proceso.

Para implementarla:

- Aplique `filtro(img)`, donde `filtro` es una función que recibe y devuelve una imagen.
- Calcule la respuesta con `cv2.morphologyEx(..., cv2.MORPH_BLACKHAT, ee)`, con un elemento elíptico de `k_bh × k_bh` creado con `cv2.getStructuringElement(cv2.MORPH_ELLIPSE, ...)`.
- Umbralice con `umbral = media(respuesta) + k_sigma · desviación(respuesta)` y devuelva `(respuesta > umbral).astype(np.uint8)`. No aplique post-proceso.

Paso 3. Construir la Tabla 1 y el recorte ampliado

- Para cada filtro de `FILTROS`, aplíquelo a las 118 imágenes y calcule el CNR medio y la desviación estándar media del fondo.
- Calcule el F1 aguas abajo con `evaluar_conjunto(lambda x, f=f: segmentar_blackhat_filtro(x, f))`. El argumento `f=f` es necesario para que cada `lambda` use su propio filtro dentro del ciclo.
- Muestre un recorte ampliado de una grieta bajo los seis filtros, por ejemplo la imagen `016` con `img[120:220, 180:330]`.

Filtro	CNR medio	σ del fondo	F1 aguas abajo
Sin filtro			
Promediador 5 × 5			
Gaussiano 5 × 5			
Mediana 5 × 5			
Mediana 9 × 9			
Bilateral (9, 50, 50)			

Tabla 1: Efecto del filtro de suavizado, promedio sobre 118 imágenes.

4. Parte 2. Tres rutas de segmentación

Implementarán tres formas distintas de segmentar la grieta. Para que la comparación sea justa, las rutas A y C usan mediana 5 × 5 y la ruta B usa gaussiano 5 × 5, que es el suavizado propio del detector de Canny [3].

Paso 4. Ruta A, `segmentar_otsu(img, k_mediana=5)`

- Suavice con `cv2.medianBlur(img, k_mediana)`.
- Umbralice con `cv2.threshold(suave, 0, 1, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)` [2]. La umbralización es inversa porque la grieta es oscura; el valor máximo 1 deja la máscara en $\{0, 1\}$.

Paso 5. Ruta B, `segmentar_canny(img, k_blur=5, t1=50, t2=150, k_cierre=5)`

- Suavice con `cv2.GaussianBlur(img, (k_blur, k_blur), 0)` y aplique `cv2.Canny(suave, t1, t2)`.
- Cierre los bordes con un elemento elíptico de `k_cierre × k_cierre`.
- Guarde el contorno cerrado (imagen 0/255) en la variable `contorno`. Su trabajo en este paso termina aquí.

El relleno del interior viene implementado en el cuaderno base, a continuación de su código:

```
# --- Entregado: relleno del interior (no modificar) ---
inundado = contorno.copy()
h, w = contorno.shape
cv2.floodFill(inundado, np.zeros((h + 2, w + 2), np.uint8), (0, 0), 255)
interior = cv2.bitwise_not(inundado)
return ((contorno > 0) | (interior > 0)).astype(np.uint8)
```

`cv2.floodFill` inunda con 255 todo el fondo conectado con la esquina (0, 0); la máscara auxiliar debe medir (alto + 2) × (ancho + 2). Al negar el resultado con `cv2.bitwise_not`, solo quedan en blanco las regiones encerradas por el contorno. La función devuelve la unión del contorno y ese interior, convertida a {0, 1}.

Paso 6. Ruta C, `segmentar_blackhat(img, k_mediana=5, k_bh=21, k_sigma=3.0)`

Llame a `segmentar_blackhat_filtro` del Paso 2 con la mediana como filtro: `lambda x: cv2.medianBlur(x, k_mediana)`.

Paso 7. Construir la Tabla 2 y la figura comparativa

- Evalúe las tres rutas con `evaluar_conjunto` y construya la Tabla 2 con `tabla()`.
- Calcule, para cada ruta, la fracción media de la imagen que marca (el promedio de `pred.mean()` sobre las 118 imágenes) y compárela con la fracción real de grieta, que es el promedio de `gt.mean()` (cerca de 0.016).
- Muestre las tres máscaras sobre una misma imagen usando `superponer()`.

Ruta	IoU	P	R	F1	F1 (tol = 2)	Fracción marcada
A · Otsu						
B · Canny						
C · Black-hat						

Tabla 2: Comparación de las tres rutas, sin post-proceso.

5. Parte 3. Post-proceso morfológico

Las máscaras de la Parte 2 tienen grietas fragmentadas y manchas pequeñas. En esta parte agregarán dos operaciones de limpieza y medirán el efecto de cada una, y de su orden.

Paso 8. Implementar `postprocesar(mascara, k_cierre=0, area_min=0, orden="cierre-area")`

- **Cierre:** si `k_cierre > 1`, aplique `cv2.MORPH_CLOSE` con un elemento elíptico de `k_cierre × k_cierre`.
- **Filtro de área (entregado):** si `area_min > 0`, elimina las componentes conexas (conectividad 8) de área menor a `area_min`. La etiqueta 0 es el fondo y no se conserva.
- **Orden:** con `orden="cierre-area"` aplique primero el cierre; con `orden="area-cierre"`, primero el filtro de área.
- **Valor no válido:** cualquier otro valor de `orden` debe lanzar `ValueError`.

El filtro de área viene implementado en el cuaderno base como función interna de `postprocesar`. Ustedes implementan `cierre(b)` y la aplicación de ambas etapas en el orden indicado:

```
# --- Entregado: filtro de área (no modificar) ---
def area(b):
    if area_min <= 0:
        return b
    _, etiquetas, stats, _ = cv2.connectedComponentsWithStats(b, connectivity=8)
    conservar = np.where(stats[1:, cv2.CC_STAT_AREA] >= area_min)[0] + 1
    return np.isin(etiquetas, conservar).astype(np.uint8)
```

Paso 9. Construir la Tabla 3 y la figura

Aplique `postprocesar` sobre las máscaras de las rutas A y C según la Tabla 3. Muestre además las cinco configuraciones de la ruta C superpuestas sobre una misma imagen.

Ruta	Post-proceso	IoU	P	R	F1	F1 (tol = 2)
A · Otsu	crudo					
	+ cierre(9)					
C · Black-hat	crudo					
	+ cierre(9)					
	+ área(50)					

Ruta	Post-proceso	IoU	P	R	F1	F1 (tol = 2)
	cierre(9) → área(50)					
	área(50) → cierre(9)					

Tabla 3: Efecto del post-proceso y de su orden.

6. Parte 4. Pipeline final

Paso 10. Implementar `pipeline_final(img)` y construir la Tabla 4

- Suavice con el filtro de mejor F1 aguas abajo en la Tabla 1 y segmente con `segmentar_blackhat_filtro`.
- Reconecte y limpie con `postprocesar(mascara, k_cierre=9, area_min=50, orden="cierre-area")`: primero el cierre y después el filtro de área, el orden que la Tabla 3 respalda.
- Construya la Tabla 4 con las tres rutas de la Parte 2 y el pipeline final, y un gráfico de barras de IoU, F1 y F1 con tolerancia.

Método	IoU	P	R	F1	F1 (tol = 2)
A · Otsu					
B · Canny					
C · Black-hat					
Pipeline final					

Tabla 4: Resumen del pipeline final frente a las rutas base.

Paso 11. Mostrar los peores y los mejores casos

Ordene las imágenes según el F1 del pipeline final con `evaluar_por_imagen(pipeline_final).sort_values("F1")` y muestre con `superponer()` los tres peores y los tres mejores casos, indicando el F1 de cada uno.

Sobre el desempeño esperado: un pipeline bien construido alcanza un F1 del orden de 0.5 y un IoU del orden de 0.35. No se descuenta puntaje por quedar por debajo si el análisis es correcto, ni se otorga puntaje extra por superarlo.

7. Análisis de resultados

En el informe, presente los resultados y su análisis. El análisis debe interpretar lo que muestran sus tablas y figuras y explicar por qué ocurre. Debe apoyarse en los números obtenidos: un análisis genérico, que no cite sus resultados, no obtiene puntaje completo.

- **Parte 1. Filtrado:** analice la Tabla 1 y el recorte ampliado del Paso 3.
- **Parte 2. Tres rutas de segmentación:** analice la Tabla 2, la fracción de la imagen marcada por cada ruta y la figura comparativa del Paso 7.
- **Parte 3. Post-proceso morfológico:** analice la Tabla 3 y la figura del Paso 9.
- **Parte 4. Pipeline final:** analice la Tabla 4, el gráfico de barras y los peores y mejores casos del Paso 11. Compare además su F1 con tolerancia con el de los métodos basados en aprendizaje: con tolerancia de 2 píxeles, en [4] el bosque estructurado de CrackForest [1] obtiene un F1 de 0.83 y una red convolucional, 0.92 (entrenados con 72 imágenes y evaluados en las 46 restantes).

8. Entregables

La tarea se realiza en pareja o de forma individual. Se entregan dos archivos, con el apellido de cada integrante en el nombre:

- **Cuaderno:** `Tarea2_Apellido1_Apellido2.ipynb`, construido sobre el cuaderno base, con todo el código y sus salidas.
- **Informe:** `Tarea2_Apellido1_Apellido2.pdf`, con los resultados y su análisis.

8.1 Cuaderno

- Todo el código va dentro del cuaderno, sin módulos auxiliares. Un cuaderno que importa código que no está en la entrega no puede evaluarse.
- El cuaderno debe ejecutarse de principio a fin sin errores (*Restart & Run All*), con la numeración de celdas correlativa y las salidas guardadas.
- El cuaderno no lleva el análisis. Las tablas y figuras del informe deben ser las que produce el cuaderno entregado.

8.2 Informe

- **Portada:** título de la tarea, nombre de los integrantes y fecha.
- **Estructura:** una sección por parte (Partes 1 a 4), cada una con sus tablas, sus figuras y el análisis pedido en la Sección 7, y una conclusión breve al final.
- **Consistencia:** los valores del informe deben coincidir con las salidas del cuaderno.
- **Código:** el informe no incluye código; para referirse a una función basta con su nombre.

9. Referencias

[1] Y. Shi, L. Cui, Z. Qi, F. Meng, Z. Chen, “Automatic road crack detection using random structured forests”, IEEE Transactions on Intelligent Transportation Systems, vol. 17, no. 12, pp. 3434–3445, 2016.

[2] N. Otsu, “A threshold selection method from gray-level histograms”, IEEE Transactions on Systems, Man, and Cybernetics, vol. 9, no. 1, pp. 62–66, 1979.

[3] J. Canny, “A computational approach to edge detection”, IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. PAMI-8, no. 6, pp. 679–698, 1986.

[4] Z. Fan, Y. Wu, J. Lu, W. Li, “Automatic pavement crack detection based on structured prediction with the convolutional neural network”, arXiv:1802.02208, 2018.

[5] A. Rose, Vision: Human and Electronic, Plenum Press, 1973.

[6] R. C. Gonzalez y R. E. Woods, Digital Image Processing, 4.ª ed., Pearson, 2018. Capítulos 3, 9 y 10.